

TEMA N° 4



TEMA 4 RESOLUCIÓN DE PROBLEMAS Y PRUEBAS DE ESCRITORIO



EN ESTE TEMA SERÁS CAPAZ DE...

1. Analizar lógicamente problemas computacionales del entorno real y estructurar soluciones paso a paso antes de escribir una sola línea de código.
2. Ejecutar pruebas de escritorio manuales y diseñar casos de prueba para detectar, diagnosticar y corregir tanto errores de sintaxis como errores lógicos.
3. Aplicar buenas prácticas de escritura, indentación y comentarios para crear código legible, profesional y fácil de mantener.



PRÁCTICA



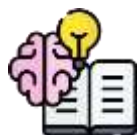
¿Qué pasa cuando el sistema de nuestro coliseo municipal cobra el doble a nuestros propios vecinos?

Imagina la siguiente situación: El CEA Herman Ortiz Camargo ha sido invitado a desarrollar un pequeño sistema de control de ingresos para el gran campeonato de fútbol que se celebrará en el coliseo municipal del Barrio 27 de Mayo. Como Técnico en Sistemas Informáticos, has diseñado un programa que registra a los equipos, cobra la inscripción y emite un boleto digital.

El día de la inauguración, llegan los equipos del Barrio 24 de Septiembre y del Barrio Residencial Norte. El sistema funciona, no se cuelga y la pantalla se ve perfecta. Sin embargo, al final del día, el tesorero de la junta vecinal nota un problema grave: el dinero recaudado en la caja no coincide con el reporte del sistema. Tras revisar rápidamente, notas que a todos los equipos del Barrio 24 de Septiembre se les cobró la inscripción multiplicada por dos, mientras que a los del Barrio Residencial Norte no se les cobró nada, a pesar de que el sistema les imprimió su boleto.

La gente está esperando respuestas, el campeonato continúa mañana, y el código tiene cientos de líneas. No hay ningún mensaje de "Error" en la pantalla de la computadora, simplemente los resultados matemáticos están mal.

¿Cómo descubres exactamente en qué parte del sistema está la falla matemática sin tener que adivinar o borrar todo el trabajo? ¿Cómo garantizas a la junta vecinal que el programa funcionará perfectamente para los equipos que llegarán mañana desde San Expedito y Las Misiones?



TEORÍA



4.1. METODOLOGÍA PARA EL ANÁLISIS LÓGICO DE PROBLEMAS COMPUTACIONALES

Antes de tocar el teclado para programar, un excelente Técnico en Sistemas Informáticos sabe que la solución a cualquier problema informático comienza en la mente y en el papel. Saltar



directamente a escribir código sin pensar suele generar sistemas frágiles y llenos de errores. Para evitarlo, aplicamos una metodología de cuatro pasos fundamentales:

1. **Comprender el problema:** Esto implica leer, escuchar y hacer preguntas. Si un comerciante del Barrio Central Fátima pide un sistema de inventario, primero debes saber: ¿Qué vende? ¿Cómo cuenta sus productos? ¿Qué datos necesita al final del día? Aquí definimos las **Entradas** (los datos que recibimos) y las **Salidas** (los resultados esperados).
2. **Planificar la solución (Diseño lógico):** En esta etapa creamos el algoritmo. Usamos pseudocódigo o diagramas de flujo para trazar el camino que transformará las entradas en salidas. Es como dibujar el plano del nuevo módulo educativo del CEA antes de poner los ladrillos.
3. **Ejecutar (Codificación):** Solo ahora traducimos nuestro diseño lógico al lenguaje de programación elegido (Python, JavaScript, Java, etc.).
4. **Revisar y Evaluar:** Consiste en probar el sistema sometiéndolo a diferentes escenarios para asegurar que cumple con el objetivo inicial.





4.2. TÉCNICAS DE TRAZADO MANUAL (PRUEBAS DE ESCRITORIO)

La **prueba de escritorio** es la herramienta de diagnóstico más confiable para un programador. Consiste en simular el comportamiento de la computadora paso a paso y a mano (generalmente en una tabla de papel), anotando cómo cambian los valores de las variables a medida que el código se ejecuta.

Imagina que tenemos un programa sencillo que cuenta cuántos estudiantes del Barrio Saó llegan temprano al CEA.

1. Dibuja una tabla en una hoja de papel.
2. Pon el nombre de cada variable en el encabezado de las columnas.
3. Añade una columna para la salida por pantalla.
4. Lee el código línea por línea, simulando ser el procesador. Tacha el valor antiguo de una variable y anota el nuevo cada vez que cambie.

Esta técnica es infalible para entender exactamente qué está haciendo el código en cada milisegundo. Si el programa dice que llegaron 10 estudiantes pero tú sabes que solo ingresaste 3, la prueba de escritorio te mostrará el momento exacto en el que la variable se sumó incorrectamente.

4.3. IDENTIFICACIÓN DE ERRORES DE SINTAXIS VS. ERRORES LÓGICOS

Cuando programamos, nos enfrentaremos inevitablemente a los "bugs" (errores informáticos). Estos se dividen principalmente en dos grandes familias:

1. **Errores de Sintaxis (Errores Gramaticales):** Ocurren cuando rompemos las reglas del lenguaje de programación. Olvidar un punto y coma, abrir un paréntesis y no cerrarlo, o escribir pirnt() en lugar de print(). La ventaja de estos errores es que la computadora se da cuenta inmediatamente y se niega a ejecutar el programa, mostrándonos una alerta roja y, por lo general, indicando la línea exacta del fallo.
2. **Errores Lógicos (Errores de Significado):** Son los más peligrosos. El código está escrito con una sintaxis perfecta, por lo que la computadora lo ejecuta sin quejarse. Sin embargo,



el resultado es incorrecto. Ocurre cuando usamos un signo de suma + en lugar de una resta -, o cuando programamos mal una fórmula matemática. El problema del coliseo en el Momento 1 fue un error lógico: la computadora hizo exactamente lo que se le ordenó, pero la orden humana estaba equivocada.

COMPARACIÓN VISUAL DE ERRORES DE PROGRAMACIÓN: SINTAXIS VS. LÓGICO

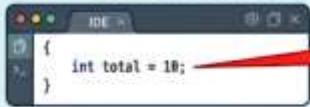
1. ERROR DE SINTAXIS

Ocurre cuando el código rompe las **REGLAS GRAMATICALES** del lenguaje.



```
public class {  
    int total = 10  
}
```

EJEMPLO: Falta de un punto y coma (;).



EL PROGRAMA NO SE EJECUTA
(El compilador no lo entiende).

ANALOGÍA
Escribir una oración sin punto final.

1. ERROR DE SINTAXIS

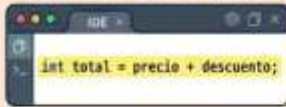
2. ERROR LÓGICO

El código es sintácticamente **CORRECTO**, pero el **RAZONAMIENTO** es erróneo.



```
int precio = 100;  
int descuento = 20;  
int total = precio + descuento;  
...
```

EJEMPLO: Usar el signo '+' (suma) en lugar de '-' (resta).



Precio: \$100 | Descuento: \$20 | Total: \$120
Resultado incorrecto: Debería ser \$80

EL PROGRAMA SE EJECUTA, PERO PRODUCE RESULTADOS INCORRECTOS
(Difícil de detectar).

ANALOGÍA
Usar sal en lugar de azúcar al cocinar.

2. ERROR LÓGICO

4.4. DETECCIÓN Y CORRECCIÓN DE ERRORES MEDIANTE CASOS DE PRUEBA

No podemos entregar un software a la población de Pailón sin antes someterlo a "Casos de Prueba" sistemáticos. Un caso de prueba es un escenario diseñado para evaluar si el sistema reacciona correctamente bajo diversas circunstancias.

Para un sistema que calcula el descuento de pasajes de tren en la estación del Barrio 24 de Septiembre según la edad, deberíamos diseñar:

1. **Casos normales:** Probar con edades comunes (ej. 25 años, 40 años).
2. **Casos extremos o límite:** Probar con los límites exactos de las reglas (ej. 0 años, 18 años, 60 años).
3. **Casos anómalos o inválidos:** ¿Qué pasa si el usuario ingresa un texto como "veinte" en lugar del número 20? ¿O si ingresa una edad negativa como -5?



Al construir una tabla con los datos ingresados, el resultado que esperamos obtener y el resultado que la computadora realmente nos da, podemos detectar y corregir vulnerabilidades antes de que el usuario final las sufra.

4.5. BUENAS PRÁCTICAS DE INDENTACIÓN Y COMENTARIOS EN EL CÓDIGO

El código no solo lo lee la máquina; principalmente lo leen otros seres humanos (incluyendo a tu "yo" del futuro). Un Técnico en Sistemas Informáticos se distingue de un aficionado por la limpieza y organización de su código.

1. **Indentación:** Es el espacio o sangría que se deja al inicio de una línea de código para indicar jerarquía. En lenguajes como Python, la indentación es obligatoria; en otros, es opcional pero fundamental para la legibilidad. Muestra qué instrucciones están dentro de un ciclo o de una condición.
2. **Comentarios:** Son textos dentro del código que la computadora ignora, pero que sirven como notas explicativas. Un buen comentario no explica *qué* hace el código (eso debería ser obvio leyéndolo), sino *por qué* lo hace.

CÓDIGO SUCIO VS CÓDIGO LIMPIO: UNA COMPARACIÓN DE LEGIBILIDAD.

CÓDIGO SUCIO Y SIN INDENTACIÓN

```
deff(n):r=1
ifn<0:returnNone
elsen>0:foriinrange(1,n+1):r=r*i
returnr
```

Difficil de Leer, Mantener y Colaborar

TIEMPO DE LECTURA Y COMPRENSIÓN PROLONGADO

CÓDIGO LIMPIO Y COMENTADO

```
1 # Función para calcular el factorial de un número n
def calcular_factorial(n):
    """
    2 Calcula el factorial de un entero no negativo n.
    3 Retorna None si n es negativo.
    4 """
    5 resultado = 1
    6 if n < 0:
    7     # El factorial no está definido para números negativos.
    8     return None
    9 elif n == 0:
    10    # El factorial de 0 es 1.
    11    return 1
    12 else:
    13    # Bucle para multiplicar desde 1 hasta n.
    14    for i in range(1, n + 1):
    15        resultado *= i # Multiplicación acumulativa
    16    return resultado
    17
    18 # Ejemplo de uso: Calcular el factorial de 5
    19 n_ejemplo = 5
    20 print(f"El factorial de {n_ejemplo} es: {calcular_factorial(n_ejemplo)}")
```

Fácil de Leer, Entender y Reutilizar

TIEMPO DE LECTURA Y COMPRENSIÓN RÁPIDO

LA LEGIBILIDAD AHORRA TIEMPO Y EVITA ERRORES.



4.6. ASISTENTES DE INTELIGENCIA ARTIFICIAL EN LA DEPURACIÓN DE CÓDIGO

En la actualidad, las herramientas basadas en Inteligencia Artificial (IA) han revolucionado la manera en que los programadores enfrentamos los problemas. Plataformas modernas pueden escanear tu código y sugerir soluciones a errores lógicos complejos.

Sin embargo, usar IA no reemplaza la necesidad de hacer pruebas de escritorio en tu mente. La IA puede proponer un bloque de código que solucione un error de sintaxis, pero si tú, como Técnico, no entiendes *por qué* ese nuevo código funciona, estarás insertando lógica desconocida en el sistema del CEA. La IA es un excelente asistente de diagnóstico (como un corrector ortográfico avanzado), pero tú eres el arquitecto responsable de la lógica del sistema.

4.7. DE LA PRUEBA DE ESCRITORIO AL TESTING AUTOMATIZADO

Las pruebas de escritorio en papel son vitales para aprender y para algoritmos cortos, pero cuando construimos sistemas grandes (como una plataforma educativa completa para todo Pailón), trazar miles de líneas a mano es imposible.

Aquí entra el "Testing Automatizado". Hoy en día, los desarrolladores escriben pequeños programas cuyo único propósito es poner a prueba el programa principal. Estas herramientas de Testing simulan miles de casos de prueba en un par de segundos. Es la evolución directa de la prueba de escritorio: primero aprendes a hacerlo tú mismo para entender la lógica, y luego le enseñas a la computadora a revisarse a sí misma.

4.8. ENTORNOS DE DESARROLLO Y DEBUGGERS VISUALES

En un entorno profesional, ya no usamos solo papel. Los Entornos de Desarrollo Integrado (IDE), como Visual Studio Code, incluyen herramientas llamadas "Debuggers" (Depuradores visuales).

Un depurador te permite establecer "Breakpoints" (puntos de interrupción). Al ejecutar tu programa, este se pausará exactamente en la línea que le indiques. A partir de ahí, puedes avanzar línea por línea presionando un botón, viendo en la pantalla cómo cambian los valores de todas tus variables en tiempo real. Es literalmente una prueba de escritorio automatizada y visual, y dominar el Debugger es una de las habilidades más cotizadas en un Técnico.



CIERRE TEÓRICO

□ Mitos vs. Realidades

Mito	Realidad
"Si el código corre y no da error de pantalla, entonces está listo."	FALSO. La ausencia de errores de sintaxis no garantiza que la lógica sea correcta. Los peores errores son invisibles y matemáticos.
"Comentar cada línea de código hace que el programa sea mejor."	FALSO. El exceso de comentarios obvios ensucia el código. Se debe comentar el <i>por qué</i> de decisiones lógicas complejas, no lo obvio.

□ Glosario de Términos

1. **Bug:** Un error, falla o defecto en un programa informático que provoca un resultado inesperado.
2. **Depuración (Debugging):** El proceso metódico de encontrar y reducir el número de bugs en un programa de computadora.
3. **Prueba de Escritorio:** Técnica de seguimiento manual, paso a paso, del flujo de ejecución de un algoritmo y sus variables.
4. **Indentación:** Sangría o espacios en blanco al inicio de una línea de código utilizados para definir la estructura y bloques lógicos.
5. **Breakpoint (Punto de interrupción):** Marca intencional que se pone en el código para que el depurador pause la ejecución en ese punto exacto.

□ Ponte a Prueba

1. ¿Qué tipo de error ocurre si escribimos la fórmula del área del triángulo como $\text{base} * \text{altura}$ olvidando dividir entre 2?
2. a) Error de Sintaxis.
3. b) Error Lógico.
4. c) Error de Indentación.



5. ¿Cuál es el propósito principal de una prueba de escritorio?
- a) Hacer que el programa corra más rápido en la computadora.
 - b) Simular manualmente la ejecución para entender cambios en variables y detectar fallos lógicos.
 - c) Instalar el software en las computadoras del módulo educativo.
6. ¿Qué característica define a una buena práctica de programación respecto a los comentarios?
- a) Explicar qué hace cada comando básico del lenguaje.
 - b) No usar comentarios para ahorrar espacio en el disco duro.
 - c) Explicar el propósito y la lógica detrás de fragmentos de código complejos.



VALORACIÓN



A veces pensamos que un error en el código es solo una molestia que nos obliga a trasnochar frente a la computadora. Pero en el mundo real, los errores de software tienen un impacto directo en la sociedad y el bienestar humano.

Piensa en los vecinos de Pailón que asisten a la posta sanitaria o realizan trámites en el gobierno municipal. Si un sistema de farmacia tiene un *error lógico* y cruza los datos de dos medicamentos distintos, la salud de una persona está en riesgo. Si un sistema bancario que gestiona microcréditos en el comercio del Barrio Central Fátima falla al hacer redondeos por falta de pruebas de escritorio exhaustivas, una familia de escasos recursos podría terminar perdiendo dinero o endeudándose injustamente.

Como Técnicos en Sistemas Informáticos, nuestra responsabilidad ética es inmensa. No somos simples "escribidores de código". Somos constructores de herramientas que gestionan datos, recursos y, muchas veces, la seguridad de las personas. Tomarse el tiempo para planificar, hacer pruebas de escritorio rigurosas y usar buenas prácticas de indentación garantiza que los sistemas que entregamos a nuestra comunidad sean confiables, transparentes y justos. La calidad de nuestro código refleja nuestro respeto hacia los usuarios que confiarán en él.



PRODUCCIÓN



Laboratorio Práctico: "El Buscador de Tarifas de Transporte Pailón"

Contexto: Un grupo de mototaxis ha solicitado un pequeño programa para calcular el costo de los viajes partiendo desde el nuevo módulo del CEA en el Barrio Saó. Tienen una tarifa base, pero se cobra un extra si el viaje es lejos (como hasta San José o Las Misiones) o si es de noche.

El código base se ha escrito en Python, pero el desarrollador anterior cometió un error lógico, errores de sintaxis y no indentó nada.

Tu Misión Paso a Paso:

1. **Analiza el problema lógico:** La tarifa base es de 5 Bs. Si la distancia es mayor a 3 km, se suman 2 Bs adicionales. Si es de noche, se suma 1 Bs extra al final de todo.
2. **Copia y ejecuta** mentalmente el siguiente código desordenado. Realiza una **Prueba de Escritorio** en una hoja de papel, usando valores de prueba: Distancia = 4 km, Noche = True.

Python

```
# CÓDIGO CON ERRORES - NO COPIAR ESTO EN PRODUCCIÓNdef
calcular_tarifa(distancia, es_noche)
tarifa_base = 5if distancia < 3: # EL requerimiento decía "
mayor a 3 km"
tarifa_base = tarifa_base + 2if es_noche == True:tarifa_base = tarifa_base - 1 #
El requerimiento decía "
se suma 1 Bs"
return tarifa_base
```

1. **Detecta y Corrige:**
 1. Encuentra el error de sintaxis en la primera línea.
 2. Encuentra los dos errores lógicos graves (pistas comentadas).
 3. Aplica las reglas de indentación correctas de Python.
 4. Añade comentarios profesionales explicando la lógica.



2. **Genera la Solución Limpia:** Escribe el código final corregido en tu entorno de desarrollo local. Aquí tienes la estructura de cómo debería verse un código profesional y limpio:

Python

```
# CÓDIGO CORREGIDO Y LIMPIO (Ejemplo de Producción)
# Esta función calcula la tarifa de transporte desde el Barrio Saó # aplicando
recargos por distancia y horario nocturno. def calcular_tarifa(distancia,
es_noche): # Corrección de sintaxis: se añadieron los dos puntos (:) #
Inicializamos la variable con el precio base acordado     tarifa_base = 5
# Corrección Lógica: Si la distancia es mayor a 3km, hay recargo     if distancia
> 3:         tarifa_base = tarifa_base + 2         # Corrección Lógica: Si es
de noche, el precio aumenta, no disminuye     if es_noche == True:
tarifa_base = tarifa_base + 1         # Retornamos el valor final calculado
return tarifa_base
# Casos de prueba solicitados:
# Viaje al coliseo (2km, de día): Debería ser 5 print(calcular_tarifa(2, False))
# Viaje a Las Misiones (4km, de noche): Debería ser 5 + 2 + 1 =
8 print(calcular_tarifa(4, True))
```

5. RECURSOS ADICIONALES

Para seguir perfeccionando tus habilidades en resolución de problemas y limpieza de código, te recomiendo explorar las siguientes herramientas y documentos oficiales:

1. **Python Tutor (pythontutor.com):** Una herramienta interactiva en línea increíble que te permite escribir código en Python, JavaScript o Java, y visualizar la ejecución paso a paso. Es literalmente una prueba de escritorio animada en tu pantalla.
2. **MDN Web Docs - Guía de Depuración (developer.mozilla.org):** Una excelente documentación oficial que te enseña a leer errores en la consola y a pensar lógicamente cuando algo falla, esencial para el desarrollo de software actual.
3. **Visual Studio Code - Debugging (code.visualstudio.com/docs/editor/debugging):** El manual oficial del entorno de desarrollo más utilizado en el mundo. Aquí aprenderás a configurar Breakpoints, vigilar variables e inspeccionar el código de manera profesional.